01 QC and preprocessing

This notebook starts from the deposited raw count object: 44,716 genes x 1,116 nuclei after population selection and before QC. It derives the sample level metadata, computes per-nucleus QC metrics, and applies the relevant gene and nucleus filters. The validated result is 19,189 genes x 1,050 nuclei.

```
suppressPackageStartupMessages({
  library(SingleCellExperiment)
  library(scuttle)
  library(Matrix)
})
source(file.path("..", "scripts", "helpers.R"))
paths <- load_paths()
dir.create(paths$work_dir, showWarnings = FALSE, recursive = TRUE)
```

Parameters

```
# Nucleus QC thresholds
min_counts <- 80000   # nCount_RNA >= 80,000
min_features <- 4500    # nFeature_RNA >= 4,500
max_mito <- 0.05    # mitochondrial fraction <= 0.05
max_ribo <- 0.05    # ribosomal fraction <= 0.05
min_spliced <- 0.01    # spliced fraction >= 0.01
# Doublet calls are not used for filtering.

# genes detected (count > 0) in more than 5 nuclei, before nucleus QC
min_gene_cells <- 5L

# Gene filters
keep_lnc_markers <- c("MALAT1", "MEG3", "NEAT1")
ig_tr_pattern    <- "^(IG[HKL]|TR[ABGD])"
bad_biotypes <- c(
  "pseudogene", "processed_pseudogene", "unprocessed_pseudogene", "miRNA",
  "transcribed_unprocessed_pseudogene", "transcribed_processed_pseudogene",
  "rRNA_pseudogene", "unitary_pseudogene", "scaRNA", "polymorphic_pseudogene",
  "transcribed_unitary_pseudogene", "IG_V_gene", "IG_D_gene", "IG_J_gene",
  "IG_C_gene", "TR_V_gene", "TR_D_gene", "TR_J_gene", "TR_C_gene",
  "IG_V_pseudogene", "translated_processed_pseudogene", "TR_V_pseudogene",
  "translated_unprocessed_pseudogene", "IG_C_pseudogene", "sRNA",
  "TR_J_pseudogene", "IG_J_pseudogene", "IG_pseudogene", "Mt_tRNA", "Mt_rRNA",
  "snRNA", "snoRNA", "rRNA", "tRNA", "misc_RNA", "TEC", "scRNA", "ribozyme",
  "antisense")

# population selection: exclude bulk, then keep the union of (2023, non-stroke), (2024,
 ↪  control) and (2024, case on two parent cDNA plates)
chapter3_case_plates_2024 <- c("duke5208_3", "duke5208_13")

# Retain the highest-quality cell_id from technical-duplicates
duplicate_keep_ids <- c(
```

```
  "CYPVFUZR", "HINVVKXY", "HRBNNNHI", "ORDSJTJL", "PXUGDGKM", "QLVKUDFE",
  "RBSPWPMP", "RDCBYJAB", "RHUTDUQK", "SAMLZGQZ", "SRUYBSVQ", "UBGFYDHH",
  "UUTLCKBQ", "VILCTEJR", "XBVOUEHE", "XIECVKLI")

# Donor diagnosis and SLC35A2 donor alias
donor_diagnosis <- c(
  mcdbose241 = "PIK3CA", mcdbose348 = "PIK3CA", dukeepi5208 = "PIK3CA",
  mcd19035 = "SLC35A2", mcdgg19035 = "SLC35A2",
  unca35 = "control", unca30 = "control", uom6543 = "control",
  uom4390 = "control", pitt13301 = "control")
slc35a2_aliases   <- c("mcd19035", "mcdgg19035")
slc35a2_canonical <- "mcd19035"

# Diagnosis-aware genotype map
genotype_map <- list(
  PIK3CA  = c(WT = "hom-ref", Var = "high-var", Het = "het", Ambiguous = "het"),
  SLC35A2 = c(WT = "hemi-ref", Var = "hemi-var"))
control_genotype <- "hom-ref"
```

Input

```
raw <- readRDS(file.path(paths$zenodo_root, "RDS", "raw_unfiltered_object.rds"))
deposited <- as.data.frame(colData(raw))

# Keep only the pre-QC input fields
input_fields <- c("cell_id", "Sample_name", "Seq_batch", "Genotype",
                  "Case_control", "Bulk_sc", "Parent_cDNA_plate_ID",
                  "is_duplicate_group_member", "spliced_counts", "total_counts")
sce <- SingleCellExperiment(
  assays  = list(counts = assay(raw, "counts")),
  colData = colData(raw)[, input_fields],
  rowData = rowData(raw)[, c("ensembl_id", "symbol", "gene_biotype", "gene_id")])
rm(raw)
stopifnot(identical(colnames(sce), as.character(sce$cell_id)))
dim(sce)
```

```
## [1] 44716  1116
```

Sample annotations

`donor_id` is `Sample_name`, with the SLC35A2 donor's two aliases merged into one ID. Genotype classes are assigned per diagnosis, and all controls are hom-ref.

```
clean_genotype <- function(x) {
  x <- trimws(as.character(x))
  x <- sub('^["""\'‘’]+', "", x)
  x <- sub('["""\'‘’]+$', "", x)
  trimws(x)
}

donor_id <- trimws(as.character(sce$Sample_name))
donor_id[donor_id %in% slc35a2_aliases] <- slc35a2_canonical

diagnosis <- unname(donor_diagnosis[match(tolower(donor_id),
 ↪  tolower(names(donor_diagnosis)))])
```

```
g <- clean_genotype(sce$Genotype)
genotype_canonical <- rep(NA_character_, ncol(sce))
is_control <- diagnosis %in% "control"
ctrl_ok <- is.na(g) | !nzchar(g) | tolower(g) %in% c("wt", "hom-ref")
stopifnot(all(ctrl_ok[is_control]))
genotype_canonical[is_control] <- control_genotype
for (dx in names(genotype_map)) {
  m   <- genotype_map[[dx]]
  idx <- which(diagnosis %in% dx)
  mapped <- unname(m[g[idx]])
  already_canonical <- is.na(mapped) & g[idx] %in% m
  mapped[already_canonical] <- g[idx][already_canonical]
  genotype_canonical[idx] <- mapped
}
stopifnot(!anyNA(diagnosis), !anyNA(genotype_canonical))

case_ct_status <- ifelse(diagnosis %in% c("PIK3CA", "SLC35A2"), "case",
                         ifelse(diagnosis %in% "control", "control", NA_character_))

sce$donor_id           <- donor_id
sce$genetic_diagnosis  <- diagnosis
sce$genotype_canonical <- genotype_canonical
sce$case_ct_status     <- case_ct_status

table(sce$donor_id, sce$genotype_canonical)
```

```
##
##               hemi-ref hemi-var het high-var hom-ref
##   dukeepi5208        0        0 102       32     137
##   mcd19035         221       91   0        0       0
##   mcdbose348         0        0  61        0      61
##   pitt13301          0        0   0        0      99
##   unca30             0        0   0        0      60
##   unca35             0        0   0        0      59
##   UoM4390            0        0   0        0      97
##   UoM6543            0        0   0        0      96
```

Dataset selection

The manuscript includes 1,116 nuclei across five sequencing lanes and two sequencing batches

```
batch  <- trimws(as.character(sce$Seq_batch))
sname  <- trimws(as.character(sce$Sample_name))
cc     <- trimws(as.character(sce$Case_control))
plate  <- trimws(as.character(sce$Parent_cDNA_plate_ID))
bulk   <- trimws(as.character(sce$Bulk_sc)) == "bulk"

retain <- (batch == "2023" & sname != "stroke") |
          (batch == "2024" & cc == "control") |
          (batch == "2024" & cc == "case" & plate %in% chapter3_case_plates_2024)
dup_member <- as.logical(as.character(sce$is_duplicate_group_member))
dup_excluded <- dup_member & !(colnames(sce) %in% duplicate_keep_ids)

stopifnot(all(!bulk & retain & !dup_excluded))
```

Gene and nucleus QC metrics

The QC metrics are computed for genes detected in more than 5 nuclei. Gene level filters are applied before computing the per-nucleus QC metrics, so that the metrics are not biased by low-quality genes.

```
counts <- assay(sce, "counts")
detected_cells <- Matrix::rowSums(counts > 0)
genes  <- detected_cells > min_gene_cells

genes_sub <- rownames(sce)[genes]
qc <- scuttle::perCellQCMetrics(
  counts[genes, , drop = FALSE],
  subsets = list(Mt = grep("^MT-", genes_sub), Rb = grep("^RP[SL]", genes_sub)))

sce$nCount_RNA   <- qc$sum
sce$nFeature_RNA <- qc$detected
sce$percent_mito <- ifelse(qc$sum > 0, qc$subsets_Mt_sum / qc$sum, NA_real_)
sce$percent_ribo <- ifelse(qc$sum > 0, qc$subsets_Rb_sum / qc$sum, NA_real_)

# Spliced fraction from the velocyto spliced / total counts
sp  <- as.numeric(sce$spliced_counts)
tot <- as.numeric(sce$total_counts)
sce$spliced_fraction <- ifelse(tot > 0, sp / tot, NA_real_)
```

Nucleus and gene filters

```
fail <- data.frame(
  low_counts   = sce$nCount_RNA < min_counts,
  low_features = sce$nFeature_RNA < min_features,
  mito         = is.na(sce$percent_mito) | sce$percent_mito > max_mito,
  ribo         = is.na(sce$percent_ribo) | sce$percent_ribo > max_ribo,
  spliced      = is.na(sce$spliced_fraction) | sce$spliced_fraction < min_spliced)
qc_keep <- !Reduce(`|`, fail)
colSums(fail)
```

```
##   low_counts low_features         mito         ribo      spliced
##           18           52            4            0            0
```

```
sym     <- as.character(rowData(sce)$symbol)
biotype <- as.character(rowData(sce)$gene_biotype)
is_bad_biotype <- biotype %in% bad_biotypes
is_ig_tr       <- grepl(ig_tr_pattern, sym)
is_lnc         <- biotype %in% "lncRNA"
is_kept_lnc    <- sym %in% keep_lnc_markers
keep_gene <- genes & ((!is_bad_biotype & !is_ig_tr & !is_lnc) | is_kept_lnc)

rowData(sce)$detected_cells_pre_cell_qc <- as.integer(detected_cells)
rowData(sce)$passed_detection_filter    <- genes

sce_qc <- sce[keep_gene, qc_keep]
dim(sce_qc)
```

```
## [1] 19189  1050
```

Verify deposited object

```
dep <- deposited[match(colnames(sce), deposited$cell_id), ]
stopifnot(
  identical(sce$donor_id, as.character(dep$donor_id)),
  identical(sce$genotype_canonical, as.character(dep$genotype_canonical)),
  identical(sce$case_ct_status, as.character(dep$case_ct_status)),
  isTRUE(all.equal(sce$nCount_RNA, as.numeric(dep$nCount_RNA))),
  isTRUE(all.equal(sce$nFeature_RNA, as.numeric(dep$nFeature_RNA))),
  isTRUE(all.equal(sce$percent_mito, as.numeric(dep$percent_mito))),
  isTRUE(all.equal(sce$spliced_fraction, as.numeric(dep$spliced_fraction))),
  identical(colnames(sce_qc), dep$cell_id[as.logical(dep$in_filtered_annotated_object)]))

ann <- readRDS(file.path(paths$zenodo_root, "RDS", "filtered_annotated_object.rds"))
stopifnot(identical(rownames(sce_qc), rownames(ann)),
          identical(colnames(sce_qc), colnames(ann)),
          identical(as.matrix(assay(sce_qc, "counts")), as.matrix(assay(ann, "counts"))))
rm(ann)
```

```
saveRDS(sce_qc, file.path(paths$work_dir, "01_qc_filtered.rds"))
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.
 ↪  framework/Versions/A/libBLAS.dylib
## LAPACK:
 ↪  /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
 ↪  LAPACK version 3.12.1
##
## locale:
## [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats4    stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
##  [1] Matrix_1.7-4                scuttle_1.20.0
##  [3] SingleCellExperiment_1.32.0 SummarizedExperiment_1.40.0
##  [5] Biobase_2.70.0              GenomicRanges_1.62.1
##  [7] Seqinfo_1.0.0               IRanges_2.44.0
##  [9] S4Vectors_0.48.0            BiocGenerics_0.56.0
## [11] generics_0.1.4              MatrixGenerics_1.22.0
## [13] matrixStats_1.5.0
##
## loaded via a namespace (and not attached):
##  [1] cli_3.6.5           knitr_1.51          rlang_1.1.7
##  [4] xfun_0.55           otel_0.2.0          DelayedArray_0.36.0
```

```
##  [7] htmltools_0.5.9     rmarkdown_2.30      grid_4.5.2
## [10] evaluate_1.0.5      abind_1.4-8         fastmap_1.2.0
## [13] yaml_2.3.12         compiler_4.5.2      codetools_0.2-20
## [16] Rcpp_1.1.1          XVector_0.50.0      BiocParallel_1.44.0
## [19] lattice_0.22-7      digest_0.6.39       SparseArray_1.10.8
## [22] parallel_4.5.2      tools_4.5.2         beachmat_2.26.0
## [25] S4Arrays_1.10.1
```